

Document Number: P1669R0

Date: 2019-06-10

Audience: Evolution Working Group, Evolution Working Group Incubator (SG17)

Reply-to: Erich Keane <<erich.keane@intel.com>>

Callsite Based Inlining Hints: `[[always_inline]]` and `[[never_inline]]`

Revision History:

R0: Initial Version.

Motivation:

While compiler implementations typically do a fantastic job with optimization particularly when it comes to inlining decisions, a sufficiently motivated developer may be able to properly benchmark their application find individual cases where they believe the compiler's decision does not provide optimal performance. Allowing the programmer to express their research in source as a hint to the optimizer is an oft-used feature.

P1465R0 proposes the commonly used function level annotation for `[[always_inline]]` and `[[never_inline]]`. However, these attributes tend to be a "big hammer" and frequently result in reduced performance. Function level attributes end up applying to all uses of a function, not just the uses that the programmer analyzed. This results in other code having interrupted vectorization, inlining of other functions, and cache-problems or enlarged functions on cold paths.

For example, one might benchmark their application and discover that the function `CloseConnection` is typically used in a case where inlining would be helpful, so they tag it with `[[always_inline]]`. However in lesser used functions, it might end up causing performance regressions, such as with the early-exit pattern:

```
bool SomethingHappened(Handle *ConnectionHandle, const Action &ThingThatHappened) {
    if (ThingThatHappened.isFatalToConnection()) { // this path is rare.
        ConnectionHandle->CloseConnection(ThingThatHappened.reason());
        return false;
    }
    // Handle that SomethingHappened here...
}
```

Since the branch is a rarely used path, inlining this is typically a bad idea. Every invocation of this function is now going to pay the price for this, as it would possibly cause code further down to end up outside of the cache. Another example where inlining might be a bad idea is in loops:

```
template<class Fun>
void ForEachConnection(vector<ConnectionHandle> ConnectionHandles, Fun Operation) {
    for (auto H : ConnectionHandles) {
        if (H->isBroken()) H->CloseConnection(IsBroken);
        Fun(H);
    }
}
```

This is another case where inlining this rarely used branch result in a significant performance regression. However, if the originally analyzed function was the following function, the programmer would have no way (or at least, no non-clunky way) to express the need to inline in one case, but not the other:

```
void CloseEachConnection(vector<ConnectionHandle> ConnectionHandles) {
    for (auto H : ConnectionHandles) {
        H->CloseConnection(IsBroken);
    }
}
```

```
}  
}
```

This function would obviously benefit strongly from inlining the call to `CloseConnection`. Yet the performance hit from `ForEachConnection` would be difficult to avoid if one were to mark the function `[[always_inline]]`. The same problem exists in the reverse, marking this `[[never_inline]]` would result in worse performance for `CloseEachConnection`. This paper proposes the solution encouraged in EWGI at the Kona 2019 meeting:

We support future call site annotation work

SF F N A SA

1 6 3 0 0

Proposal:

This paper proposes an extension to P1465R0 to permit the inlining hint attributes to appear at the statement level, where it would operate only on the calls contained in that statement. The statement level hint would override the function level attribute, permitting the programmer to better control their inlining hints. This would permit (at least) three different ways of solving the `ForEachConnection` vs `CloseEachConnection` problem listed above. The programmer could annotate `CloseConnection` with `[[always_inline]]`, then use the statement level `[[never_inline]]` on the call in `ForEachConnection`. The programmer could alternatively mark `CloseConnection` with `[[never_inline]]` and the call in `CloseEachConnection` with `[[always_inline]]`. Finally, the programmer could simply annotate all call sites with the proper attribute.

While this proposal is new to the C++ committee, it is not a novel idea, the Intel C++ Compiler[INTC_INLINE] implements this feature with an additional option. ICC provides an option for recursive as an option to instruct the compiler to either inline all calls down the callgraph of that call instruction.

Implementation experience and user feedback has lead the author to come to the conclusion that this is an overly general of a solution to the problem, and instead requests feedback on having the two take a depth parameter. Said parameter would instead be a positive (non-zero) integer that instructs the compiler to inline up to a certain depth. Controlling the depth exactly isn't implementable, since the callee may have already had inlining done, however having this be a minimum depth is possible since further inlining can be done on the top-most caller.

While this is implementable in ICC since it can do top-down inlining, the author is unsure whether GCC/Clang/MSVC can be configured with that inliner approach.

Requested Guidance

1. Do we approve of this direction?
2. Should this attribute apply to only the first call in the statement, or all? Meaning, what does `[[always_inline]]foo(bar());` do? Should `always_inline` only apply to the call to `foo`, or also the call to `bar`?
3. What do we think of the depth parameter?

References:

[P1465R0] "Function optimization hint attributes `[[always_inline]]`, `[[never_inline]]`, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1465r0.pdf>

[INTC_INLINE] "Intel C++ Compiler Developer Guide and Reference inline noinline forceinline", <https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference-inline-noinline-forceinline>